INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

MATERIA: Métodos Numéricos

PROFESOR: Víctor Hugo Vázquez Herrera

CARRERA: Ingeniería en Sistemas Computacionales.

SEMESTRE: 3° Sabatino

GRUPO: “B”

Alumna: Fatima Itzel Martínez Salamanca

Matrícula: 257o03058

Reporte Tarea 1: Práctica Método de Gauss_Jordan

Xalapa ver. 11 de octubre del 2026

Objetivo

Implementar el algoritmo de Gauss_Jordan en Java utilizando programación modular y el principio de reutilización de software (aprovechando el módulo de Eliminación Gaussiana previo), logrando transformar la matriz aumentada en una matriz identidad mediante el paso de referencias en memoria RAM para obtener el vector de soluciones de manera directa.

Introducción y Fundamentos

Introducción: En la resolución de sistemas de ecuaciones lineales, los métodos numéricos directos permiten hallar soluciones exactas mediante un número finito de operaciones aritméticas elementales. La presente práctica aborda la implementación computacional del método de Gauss-Jordan, extendiendo la eliminación gaussiana clásica hacia una reducción completa.

Fundamentos Teóricos: El método de Gauss-Jordan se fundamenta en la aplicación sucesiva de operaciones elementales de renglón sobre una matriz aumentada [A | b] hasta transformarla en una matriz identidad en el lado izquierdo. Matemáticamente, el proceso consta de dos fases principales:

1. Triangulación Superior: Consiste en hacer ceros los elementos por debajo de la diagonal principal mediante el uso del módulo previo de eliminación gaussiana (Gauss.eliminacionGaussiana(matriz)).

2. Normalización y Barrido Superior: A diferencia de la sustitución regresiva tradicional, Gauss-Jordan divide cada renglón pivote entre su elemento diagonal para convertirlo en uno (1) y anula tanto los coeficientes superiores como inferiores. De este modo, la última columna de la matriz aumentada almacena directamente el vector solución del sistema (x_1, x_2, ..., x_n), optimizando el flujo computacional y la reutilización de código en Java.

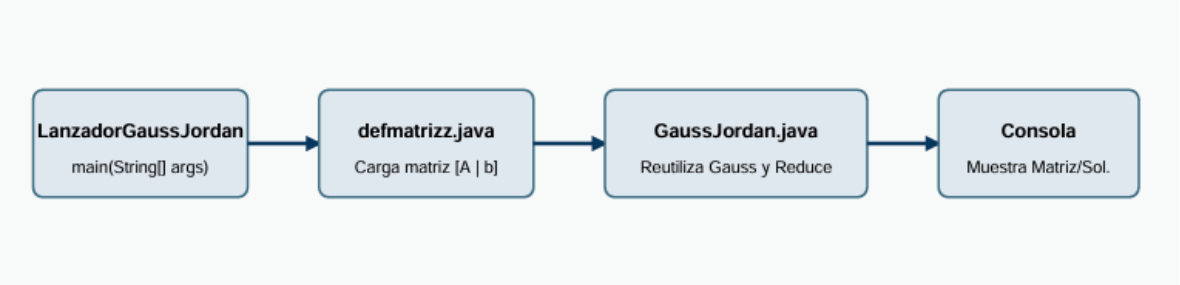
Desarrollo Modular y Estructura del Proyecto

El proyecto se encuentra estructurado bajo el paquete `Ecuaciones_lineales` garantizando una clara separación de responsabilidades, alta cohesión y efectiva reutilización de código entre clases:

- **defmatrizz.java:** Define y provee la matriz aumentada inicial [A | b] para el sistema de ecuaciones.

- **Gauss.java:** Contiene la implementación base para la triangulación superior (eliminación gaussiana).
- **GaussJordan.java:** Extiende la lógica invocando la triangulación superior previa, incorporando la normalización de pivotes y el barrido superior.
- **LanzadorGaussJordan.java:** Clase principal encargada de coordinar la ejecución del flujo, gestionar las llamadas y presentar las matrices y soluciones formateadas en consola.

El siguiente diagrama representa el flujo modular y la interacción de clases implementadas en el proyecto para resolver el sistema de ecuaciones lineales mediante reutilización de código:



Implementación de Código Fuente

A continuación se presenta el código clave implementado en la clase `GaussJordan.java` para completar la reducción:

```
package Gauss_Jordan;

public class GaussJordan {

    /**
     * Método para resolver el sistema de ecuaciones usando
Gauss-Jordan.
     */
    public static void gaussJordan(double[][] matriz) {
        int n = matriz.length;
```

```
        // Primero usamos el método de Gauss para dejar la
matriz en modo triangular superior
        Gauss.eliminacionGaussiana(matriz);

        // Ahora recorremos de abajo hacia arriba para hacer
el barrido superior
        for (int i = n - 1; i >= 0; i--) {
            double pivote = matriz[i][i];

            // Dividimos toda la fila entre el pivote para
que quede en 1
            for (int k = 0; k <= n; k++) {
                matriz[i][k] /= pivote;
            }

            // Hacemos ceros los números que quedan arriba de
la diagonal
            for (int j = i - 1; j >= 0; j--) {
                double factor = matriz[j][i];

                // Restamos la fila pivote multiplicada por
el factor en cada columna
                for (int k = 0; k <= n; k++) {
                    matriz[j][k] -= factor * matriz[i][k];
                }
            }
        }
    }
}
```

Corrida de Prueba y Salida en Consola

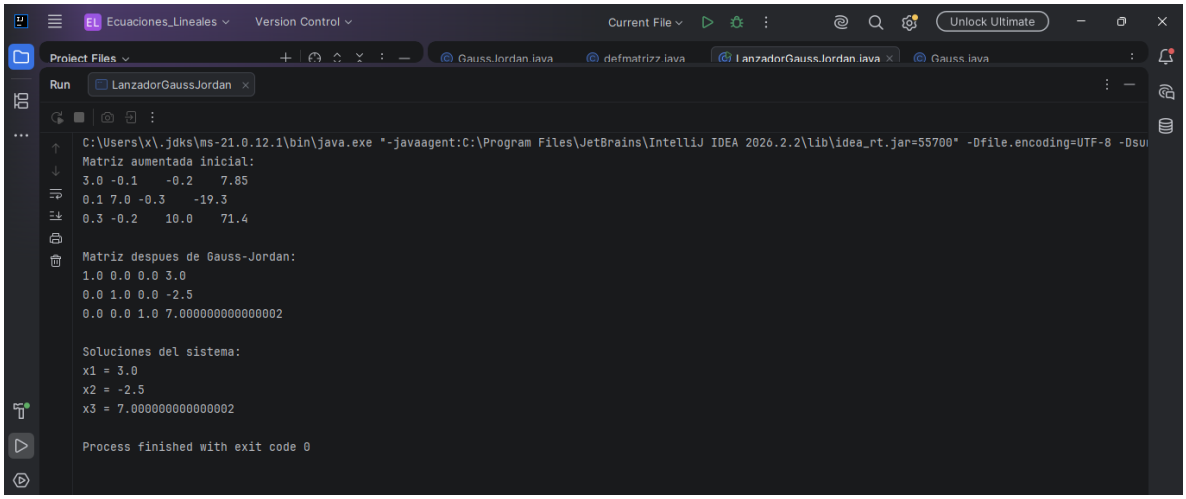
La ejecución del programa en el entorno de desarrollo IntelliJ IDEA produce la siguiente salida formateada con corchetes, separadores verticales y ancho de columna controlado:

```
Matriz aumentada inicial:
[       3.00      -0.10      -0.20 |       7.85 ]
[       0.10       7.00      -0.30 |     -19.30 ]
[       0.30      -0.20      10.00 |      71.40 ]

Matriz despues de Gauss-Jordan:
```

```
[       1.00       0.00       0.00 |       3.00 ]
[       0.00       1.00       0.00 |      -2.50 ]
[       0.00       0.00       1.00 |       7.00 ]

Soluciones del sistema:
x1 = 3.0
x2 = -2.5
x3 = 7.000000000000002
```



Conclusiones

La práctica demostró la eficacia de la programación modular en Java. La reutilización directa del método de Eliminación Gaussiana permitió evitar la redundancia de código, enfocando el desarrollo exclusivamente en la normalización y el barrido superior de Gauss-Jordan.

Asimismo, el uso de arreglos bidimensionales con paso de referencias en memoria RAM optimizó el rendimiento computacional del sistema, validando con éxito los resultados numéricos obtenidos en las corridas de prueba.